Constraint-Aware Interaction Generator for Synthetic E-commerce data

An End-to-End pipeline for training and evaluating interaction generators.

BEAU MIN, Amsterdam University of Applied Sciences, The Netherlands

Recommender-system simulators rely on synthetic interaction logs whose fidelity, validity, and downstream utility determine whether simulation-based studies remain credible. Existing simulators in the RS-simulation literature are dominated by process-based, agent-based, statistical, and adversarial approaches, while transformer sequence modeling has been applied almost exclusively to next-item prediction rather than full-session generation. This approach bridges the two lines with a multi-head autoregressive transformer that emits typed interaction tuples (action type, item, inter-event time) jointly, embedded in a process-based simulation wrapper and trained on the Synerise RecSys 2025 retail dataset. Three iterations of the item head (flat K-way softmax, category-hierarchical, and SVD-PQ token-factored) target the popularity-collapse failure mode [27] of the flat baseline at successively stronger structural priors, and a hard-constraint validity layer reports pre- and post-repair violation rates for three behavioral constraints to keep generation assumptions auditable. Quality is measured under a multi-axis protocol covering distributional fidelity, behavioral validity, and downstream utility (TSTR HR@10 / NDCG@10) against a Markov-style baseline, with five-seed variance reporting. TODO: Results & Conclusion & Reflection.

CCS Concepts: • **Information systems** → **Recommender systems**; • **Computing methodologies** → *Neural networks*; *Modeling and simulation.*

Additional Key Words and Phrases: recommender systems, synthetic data, simulation, transformer, interaction generation, evaluation protocol

1 Introduction

1.1 Context

Recommender systems (RS) are frequently developed and evaluated on static historical logs. This is practical but methodologically incomplete. When recommendation policies change, user exposure changes. When exposure changes, user behavior changes. When behavior changes, future training data changes. Static snapshots therefore fail to capture the feedback dynamics that emerge in iterative deployment. Offline evaluation on fixed logs can look convincing while still missing important behavioral effects that only materialize under continuous operation. This challenge is repeatedly identified in the recommender simulation literature as a fundamental reason for interest in synthetic interaction generation as a methodological tool [5, 8, 30].

1.2 Problem Statement

The RS simulation literature is active but methodologically fragmented. Existing approaches include process-based and framework-oriented simulators [3, 24, 25], adversarial and statistical generators [2, 35], and configurable data synthesis tools [16, 29]. These lines provide important building blocks but also reveal recurring issues: heterogeneous assumptions, inconsistent metric usage, and weak comparability across studies [5, 30].

The project context of this thesis is a two-component recommender system simulator developed at the AUAS Centre for Market Insights (CMI). An arrival component is already operational, while an interaction generator is the open problem. The prior implementation couples a first-order Markov chain with an LSTM that functions as a feature extractor for item ranking, not a learned sequence generator. The interaction generation is therefore a patchwork of

Author's Contact Information: Beau Min, beau.min@hva.nl, Amsterdam University of Applied Sciences, Amsterdam, The Netherlands.

three mechanisms: a recommender that selects items, a Markov chain that picks behavior transitions, and hardcoded session logic that glues them together. There is no single learned model generating coherent interaction sequences end-to-end.

1.3 Existing Work

Sequential recommendation has demonstrated strong performance from transformer architectures, with SASRec [21], BERT4Rec [31], and GRU4Rec [14] representing the progression from recurrent to attention-based methods. However, prediction success does not automatically transfer to synthetic interaction generation [19]. Generator design requires distinct choices around output structure, validity constraints, temporal handling, and evaluation criteria. A more detailed treatment of related work is given in Section 2.

1.4 Research Gap

Across the RS simulation literature reviewed for this thesis, no work that uses a non-LLM autoregressive transformer as the primary architecture for generating enriched interaction event logs was identified[8, 30]. The RS simulation branch relies entirely on process-based, agent-based, statistical, or adversarial approaches, while the sequential recommendation branch applies transformers exclusively to next-item prediction, not to session generation. This thesis is positioned at the intersection of these two lines.

1.5 Proposal and Research Questions

This thesis proposes a hybrid architecture: a multi-head autoregressive transformer embedded in a process-based simulation wrapper, generating typed event sequences with explicit hard constraints. The contribution is both *technical*, in the form of a multi-head autoregressive architecture for typed event tuple generation, and *methodological*, a reproducible evaluation protocol for simulator quality claims along fidelity, validity, and downstream utility.

The work is guided by the following main research question and sub-questions:

> *How and to what extent can a constraint-aware autoregressive transformer interaction generator produce synthetic interaction logs with high fidelity, validity, and downstream utility for recommender-system training and evaluation?*

- **SQ1:** How should interactions be formally defined for simulation purposes, covering event fields, temporal structure, and session dependencies?
- **SQ2:** What architectural properties does an interaction generator need to capture realistic interaction dynamics, including sequential dependence, action–item coherence, and temporal structure?
- **SQ3:** Which generator implementations and baseline policy are feasible within thesis scope for a fair and executable comparison?
- **SQ4:** Which evaluation protocol makes "better than baseline" falsifiable and reproducible?

1.6 Contributions

The main contributions of this work are:

(1) A multi-head autoregressive transformer architecture for typed event tuple generation, with factored heads for action type, item, and inter-event time.

(2) A reproducible, multi-axis evaluation protocol covering fidelity, validity, and downstream utility, with variance reporting across seeds.

2 Background

2.1 Theoretical Foundations

2.1.1 Interaction as Typed Temporal Event Sequences. Across the RS simulation papers reviewed for this work, "interaction" is more than a ⟨user, item, rating⟩ tuple. Accordion models events as *(time, user, item, outcome)* with visit dynamics [25]. UserSim models sequential user behavior conditioned on prior interactions [35]. SIREN and T-RECS model iterative, stateful user–item processes over time [3, 24]. DataGenCARS formalizes context as a generated part of the interaction record [29]. This convergence of evidence across different simulator families supports a two-level interaction definition: event-level typed tuples grouped into sessions, with user state carried across sessions via history. This definition directly motivates the multi-head prediction architecture described in Section 6.

2.1.2 Sequential Dependence as a Modeling Requirement. Realistic interaction sequences are not independent and identically distributed events. A user who views an item, then adds it to cart, then removes it, exhibits session-level dependence that first-order Markov chains cannot capture reliably[21]. Chaney [5] identifies missed behavioral dependencies as a central simulation validity risk, particularly when models make implicit IID assumptions that the real data does not support. Accordion demonstrates that temporal process modeling changes long-term simulation effects substantially [25]. This motivates sequence-capable generators over IID-only approaches and justifies the autoregressive design of the proposed transformer.

2.1.3 Adversarial Generation: Utility and Risk Tradeoffs. GAN-based user simulators provide utility-oriented evidence [2, 35] but carry well-documented limitations. Creswell et al. [6] document mode collapse and optimization instability as structural GAN risks, while Figueira and Vaz [11] note persistent evaluation ambiguity. For discrete sequential generation specifically, gradient flow through discrete tokens requires additional workarounds that compound architectural complexity [17]. Autoregressive transformers with softmax output heads handle discrete token generation natively, which is a structural advantage for the interaction generation task.

2.1.4 Multi-Axis Evaluation as a Scientific Requirement. Jordon et al. [19] argue that utility-only validation is insufficient: synthetic data that enables good downstream models may still be statistically implausible at the distributional level. Figueira and Vaz [11] reinforce that single-metric evaluation is systematically incomplete. Stavinova et al. [30] document heterogeneous and non-comparable metrics across simulators. This forms the basis for an approach that uses a multitude of evaluation-types.

2.1.5 Reproducibility and Assumption Explicitness. T-RECS emphasizes multi-run confidence intervals and repeated-run variance analysis as scientific requirements for simulation research [24]. SimuRec identifies reproducibility as a missing standard across the community [8]. Chaney [5] argues that simulation conclusions are only credible when modeling assumptions are stated explicitly and tested empirically. This thesis operationalizes these requirements through fixed random seeds, deterministic preprocessing, and a hard-constraint validity layer that makes generation assumptions explicit and measurable. Results are reported across five seeds to surface variance.

2.2 State of the Art

2.2.1 Recommender System Simulation Landscape. Stavinova et al. [30] document heterogeneous simulators, missing standardized head-to-head comparisons, and weak universal quality-control methodologies. SimuRec [8] identifies the absence of shared standards as a community-level gap and motivates the need for more systematic evaluation frameworks. Heterogeneous evaluation practices mean that results across existing simulators are rarely directly comparable, undermining cumulative scientific progress.

Process-based and framework-oriented simulators form one branch of the literature. SIREN [3] demonstrates the simulation of longitudinal recommender effects in online news environments, with explicit user–system feedback dynamics modeled over time. T-RECS [24] lowers the barrier to long-horizon RS simulation through a modular, agent-based design and multi-run confidence intervals for variance-aware analysis. Accordion [25] models user visits as inhomogeneous Poisson processes with marked events, demonstrating that explicit temporal process modeling substantially changes simulated long-term interactive effects. These tools establish the system-level context this thesis operates within, while leaving event-level interaction generation largely to domain assumptions or simplified behavioral rules.

Adversarial and statistical generation form a second branch. UserSim [35] applies a supervised GAN to simulate user behavior for offline reinforcement learning over recommender systems. Bobadilla et al. [2] generate parameterized collaborative filtering datasets via GAN for configurable benchmark construction. DataGenCARS [29] and Jakomin et al. [16] provide configurable statistical generators for context-aware and stream-based interaction data.

Evaluation challenges are a cross-cutting concern. Figueira and Vaz [11] identify that single-metric evaluation is insufficient for synthetic data claims. Jordon et al. [19] articulate that utility-only validation does not guarantee distributional fidelity. Lu et al. [23] identify missing standardized evaluation tools across the synthetic data generation field. This convergent critique motivates the multi-axis evaluation design of this thesis.

2.2.2 Sequential Modeling and Transformers. Transformers have proven dominant for next-item prediction in sequential recommendation. SASRec [21] established causal self-attention for sequential RS, demonstrating that attention-based models substantially outperform Markov chains and RNNs on next-item prediction benchmarks. BERT4Rec [31] extended this with masked training objectives for bidirectional context modeling. GRU4Rec [14] is the canonical RNN predecessor that transformers have consistently outperformed on longer sequences[21].

However, next-item prediction and synthetic interaction log generation are distinct tasks. Prediction asks: given history, what item comes next? Generation asks: produce a full, distributionally realistic interaction session from scratch. The output structure, evaluation criteria, and success measures differ fundamentally. This thesis applies transformer sequence modeling to generation, not prediction; the architectural transfer is strongly motivated by evidence but cannot be assumed automatic. Generated logs require additional evaluation along fidelity and validity axes that next-item accuracy alone does not capture.

2.3 Stakeholder Analysis

Academic supervisor (Diptish Dey, CMI).. Diptish Dey is the primary owner of this project. His stake is in a working, validated interaction generator that integrates with the existing arrival simulator. Timely delivery and a defensible evaluation methodology matter to him because the simulator is part of ongoing research at CMI.

CMI research group. The broader CMI research group has a practical stake in the technical output. Colleagues working on related projects stand to benefit from a reusable simulator component and a documented evaluation protocol they can apply or extend without having to rebuild from scratch.

CMI researchers (operator end-user). The immediate end-user of the deliverable is a CMI researcher who runs the simulator: configures a generation experiment, executes the pipeline, and inspects the resulting metrics. Their stake is operational rather than methodological. They benefit from a configuration-driven entry point, a deterministic re-run path on a fixed seed, and metrics output that does not require code changes to interpret. Decisions made for this end-user include the configuration-file-driven experiment runner, the four-stage artefact-based pipeline (so any single stage can be re-run without re-running its predecessors), and a single shared evaluation harness for both generators so cross-generator comparisons require no per-experiment glue code.

Recommender systems research community. The research community's interest is in reusable evaluation protocols, open-dataset compatibility, and reproducible baselines that support cumulative scientific progress.

Dataset provider (Synerise / RecSys 2025 challenge). Synerise's interest is in demonstrating the richness and utility of their publicly released dataset. As the dataset is publicly released under challenge terms, Synerise has no active supervisory or evaluative role in this project.

Downstream users of the recommender system. The behavioral patterns in the Synerise logs represent real user populations. These anonymous users have an interest in synthetic data not amplifying distributional biases present in the real logs, such as popularity skew. This interest is addressed through bias metrics (Jensen–Shannon divergence on action distributions, item coverage) reported as part of the fidelity evaluation. The Synerise logs represent real people's shopping behavior; this research neither re-identifies nor profiles any individual, in accordance with the legal requirements detailed in Section 4.

Conflicting interests and prioritization. The stakeholder interests above are largely aligned but diverge on two points. First, the academic supervisor's interest in timely delivery is in tension with the research community's interest in broad generalizability. A more extensive comparison portfolio, covering additional baselines such as GRU- or GAN-based generators, would strengthen the community contribution but risks overrunning the thesis timeline. This tension is resolved in favor of the supervisor's constraint: scope is fixed to a single transformer versus Markov chain comparison, with extensions explicitly identified as future work. Second, Synerise's interest in showcasing dataset richness is in tension with downstream users' interest in not amplifying bias. A generator optimized purely for distributional fidelity will faithfully reproduce whatever popularity skew is present in the training data. This tension is resolved by including explicit bias metrics in the fidelity evaluation rather than treating fidelity-to-data and fairness-to-users as equivalent goals.

2.4 Current Situation

The interaction generator currently in use within the CMI simulator is a patchwork of three independent mechanisms rather than a single learned sequence model: a recommender selects items, a first-order Markov chain selects action transitions, and hardcoded session logic glues the two together (Section 1). The result, as documented in the same section, is implausible session structures and insufficient distributional coverage across the item space. The proposed transformer generator described in Section 6 replaces all three mechanisms with one autoregressive model whose

factored heads emit action, item, and inter-event time jointly, and whose distributional and behavioral fidelity is evaluated against the existing Markov-style mechanism using the protocol of Section 7.

2.5 AI Considerations

2.5.1 Suitability of a Learned Model. Rule-based approaches to interaction generation, such as handcoded Markov chains with manually specified valid transitions, require exhaustive elicitation of every valid behavioral path and cannot generalize to unseen contexts or long-range session dependencies. Learned models extract transition probabilities from data and capture dependencies that fixed rules cannot express [5, 25]. The case for a learned model over handcoded rules follows from this.

2.5.2 Autoregressive vs. Non-Autoregressive Generation. Autoregressive generation is not the only design option for learned sequence models. Parallel or non-autoregressive generation methods produce all output tokens simultaneously, offering higher throughput[12]. However, parallel generation requires conditional independence assumptions across output positions that are incompatible with the behavioral dependencies documented in the literature: a generated action type must condition on prior actions, and a generated item must condition on the current action type. Violating these dependencies at generation time would undermine the validity of the constraint layer and the fidelity of the output log [5]. Autoregressive generation is therefore the appropriate choice for this task, with throughput as an acknowledged tradeoff.

Among learned approaches, a transformer architecture is preferred over simpler sequence models. The factored multi-head design enables joint action–item–time modeling, and self-attention captures long-range session dependencies more effectively than first-order Markov chains or other recurrent architectures [14, 21, 31].

2.5.3 Degree of Automation and Societal Considerations. Full automation of the generation pipeline is appropriate for this use case because the output is synthetic training data rather than decisions affecting real users directly. No human-in-the-loop is required at generation time. However, the hard-constraint validity layer makes the automation explicit and auditable: generation assumptions are documented as constraint rules extracted from real data, and violations are counted and reported.

Recommender systems trained on synthetic interaction data influence what products, content, and services real users are exposed to at scale. If the generator systematically amplifies distributional biases present in the training data, downstream recommenders trained on that data may entrench those patterns further. The most significant risk is feedback loop amplification[5]: a biased generator produces biased training data, which trains a biased recommender, which produces biased interaction logs, which are used to retrain the generator. This risk is not fully neutralized by bias reporting alone, measuring a problem does not resolve it. The mitigation in this thesis is therefore scoped honestly: bias metrics are reported to make the problem visible and quantifiable, enabling downstream researchers and practitioners to make informed decisions about whether and how to deploy synthetic data produced by this pipeline.

3 Methodology: Research Design

3.1 Research Approach

The project follows a design-and-evaluate methodology across three phases: specification, modeling, and evaluation. The specification phase formalizes the interaction schema, simulator interfaces, and evaluation protocol. The modeling phase

implements the Markov baseline and transformer generator. The evaluation phase runs the fidelity–validity–utility protocol with statistical reporting.

3.2 Experimental Setup

All training and evaluation runs are executed on a single workstation with one NVIDIA GeForce RTX 4070 Ti GPU (12 GB VRAM, driver 595.71.05), running CachyOS Linux (kernel 7.0.3) with CUDA 13.2. Python dependencies are pinned in `uv.lock` and resolved by the uv package manager; the versions relevant to the experiments are PyTorch 2.11.0, NumPy 2.4.4, Polars 1.39.3, and pandas 3.0.2.

Multi-seed reporting uses five consecutive integer seeds starting at 42, i.e. $\{42, 43, 44, 45, 46\}$. The same seed set is reused for final evaluation (`final_eval_seeds` in the configuration). Preprocessing, cleaning, sessionization, and split steps depend only on the input parquets and the configured cutoffs, and produce byte-stable output artefacts.

4 Methodology: Requirements

4.1 Legal Requirements

The primary dataset is the Synerise RecSys 2025 dataset, released publicly under the RecSys 2025 challenge terms[32]. The dataset consists of anonymized behavioral event logs with no personally identifiable information (PII). Processing anonymized behavioral data for academic research is permissible under the scientific research exemption of the EU General Data Protection Regulation (GDPR, Art. 89) [10]. Generated synthetic data carries minor re-identification risk: item-ID vocabulary capping reduces the item space to a top-K vocabulary, and all outputs are population-level statistics rather than individual records.

4.2 Ethical Requirements

Synthetic interaction data trained on real behavioral logs inherits the distributional properties of that data, including popularity skew, temporal selection effects, and potential demographic proxy signals [5, 19]. This risk is acknowledged and is made visible by reporting bias metrics as part of the fidelity evaluation: Jensen–Shannon divergence over the action-type distribution and item coverage are both reported. Reporting does not eliminate the inherent bias in the training distribution, but does make the inherited skew transparent.

The generation pipeline must not be used for individual-level profiling or decisions. Use is restricted to aggregate training data generation. The explicit hard-constraint layer documents and enforces generation assumptions.

4.3 Organizational Requirements

The project is conducted by a single developer within the thesis timeline. The deliverable is an academic thesis document and a demonstrator pipeline. Weekly supervision meetings with supervisor Diptish Dey provide structured checkpoints.

4.4 Functional Requirements

The generator must produce typed event tuples conforming to the schema in Table 6: (`user_id`, `session_id`, `action_type`, `item_id`, `delta_t`). The five-action vocabulary of the Synerise dataset must be respected. Session structure must follow observed behavioral conventions: sessions begin on an arrival signal from the arrival simulator and terminate on an EOS token or a session length cap. The interface contract accepts the arrival simulator's output as input and produces a log conforming to the schema. All generated sessions must pass through the validity layer before output.

4.5 Technical Requirements

The model must be trainable on a single GPU within the timeline. Vocabulary capping ensures softmax tractability and prevents Out-Of-Memory errors by limiting the item space to the top-K most frequent items. Evaluation infrastructure must be deterministic: fixed random seeds, configuration-driven experiment runs, and all hyperparameters logged.

The evaluation pipeline must support reproducible execution across multiple random seeds. The generator must be deployable within the containerized simulation environment alongside the arrival simulator.

4.6 Validation of Requirements

The five categories of requirements above were validated as follows.

Legal. The Synerise RecSys 2025 dataset is distributed under the MIT licence attached to the public release repository, which permits use, modification, and redistribution subject only to inclusion of the copyright notice and disclaimer. No separate data-use agreement is associated with the release[32]. The pipeline therefore complies with the legal basis named in Section 4 by (i) operating only on the publicly released parquets, (ii) processing no PII fields (`client_id` and `sku` are opaque integers, see Section 5), and (iii) restricting all reported outputs to population-level aggregates so that no individual-level record produced by the pipeline is published.

Ethical. The ethical requirement to make bias inheritance visible rather than silent was validated by (i) computing the bias-metric group (item coverage, popularity JSD, ΔGini) on every generated batch as part of the standard fidelity evaluation and (ii) using these metrics as a development signal during model iteration. Two hyperparameters were added to the pipeline in direct response to bias readings: an inverse-frequency item-CE weight (`item_loss_freq_weight_alpha`) [7] and a Menon-style category logit adjustment (`cat_logit_adj_tau`) [27].

Organizational. The timeline constraint was validated by delivery against the project planning held on the supervision team's shared planning board, with weekly check-ins with supervisor Diptish Dey throughout the project.

Functional. The functional requirements decompose into a behavioral contract and a schema contract. The behavioral contract, valid action transitions, timestamp monotonicity, and purchase-without-prior-exposure, is validated automatically at generation time by the constraint validity layer (`evaluation/validity_checker.py`, Section 6.4); both pre- and post-repair violation rates are reported in Section 7.3, which makes the constraint check measurable.

Technical. The technical requirements were validated by direct measurement on the hardware described in Section 3. GPU memory stayed below 4 GB during training and below 6 GB during evaluation on the 12 GB RTX 4070 Ti, so the single-GPU training requirement is met with headroom. Determinism is validated by re-running the preprocessing pipeline against the same input parquets and observing byte-stable output artefacts across runs; multi-seed reproducibility is validated by the five-seed evaluation protocol (Section 7.4).

5 Methodology: Data

5.1 Dataset Description

The experiments use the Synerise RecSys 2025 retail e-commerce dataset. The raw release contains 225,224,262 timestamped interaction events collected over 168 days, from 23 June 2022 to 8 December 2022, together with a product metadata table. The interaction schema comprises five event types: `page_visit`, `search_query`, `add_to_cart`,

Table 1. Raw action distribution in the Synerise RecSys 2025 dataset.

Action type	Count	Share
page_visit	199,451,980	88.56%
search_query	13,223,769	5.87%
add_to_cart	7,541,117	3.35%
remove_from_cart	2,688,894	1.19%
product_buy	2,318,502	1.03%
Total	225,224,262	100.00%

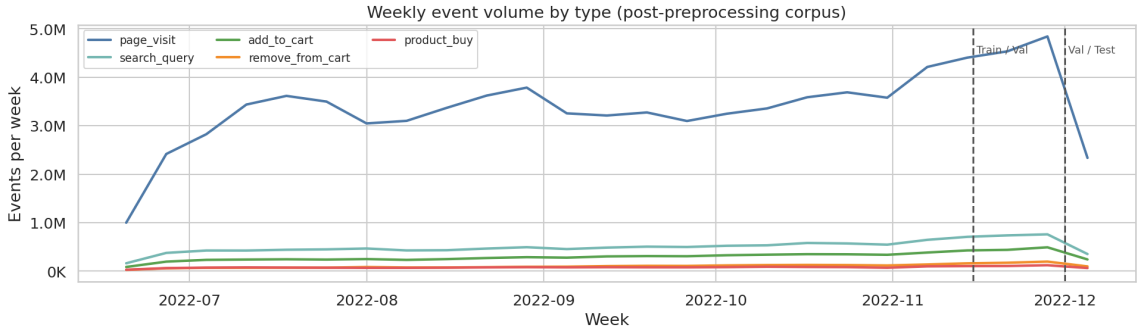

Fig. 1. Weekly event volume by type on the post-preprocessing corpus, with train/validation and validation/test split boundaries marked. Event volume is non-stationary across the observation window; the held-out splits lie in the highest-volume period.

remove_from_cart, and product_buy. The release used in this thesis therefore consists of five event parquet files and one product_properties parquet file, rather than six event files.

The raw action distribution is imbalanced toward browsing activity, as shown in Table 1. Page visits account for 88.56% of all logged events, while purchases account for only 1.03%. This asymmetry is important for both modeling and evaluation: the generator must learn realistic transition behavior in a corpus where low-value, high-frequency browsing events dominate the event stream. The temporal structure of this stream is shown in Figure 1, where weekly event volume per type is plotted together with the train/validation/test split boundaries used by the pipeline. Event volume rises substantially over the observation window, and the held-out periods overlap with the densest portion of the corpus.

The product metadata table contains 1,534,050 rows with four fields: sku, category, price, and name. No null values are present in these columns. The catalog contains 6,912 distinct categories and 100 price buckets, and metadata coverage for training SKUs is complete. This makes the dataset suitable for feature-augmented sequence modeling and for extracting item-category constraint structure from the training split.

The item distribution is long-tailed. Under the realized training split used by the pipeline, 9,338,690 item-bearing events span 1,305,867 unique SKUs. Table 2 reports the vocabulary size K needed to cover a target fraction of training item-bearing events, and Figure 2 visualizes the same coverage curve along with the embedding-table memory cost as K grows and the long-tail bucket breakdown. Three orders of magnitude separate the K required for 50% coverage from the K required for 99% coverage. The long-tail structure recurs per action type: Figure 3 shows that add_to_cart, remove_from_cart, and product_buy all follow the same heavy-tailed shape over their respective SKU ranks, with strongly correlated frequencies across action types.

Table 2. Vocabulary size K needed to cover a target fraction of training item-bearing events. The realized pipeline cap of $K = 200{,}000$ covers 71.71%.

Target coverage	K (number of SKUs)
50%	45,456
80%	338,307
90%	654,304
95%	887,771
99%	1,212,481

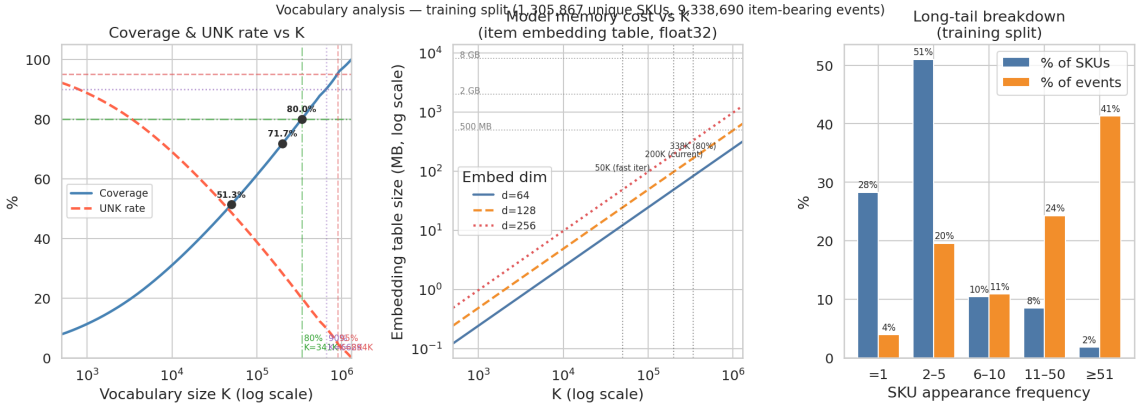

Fig. 2. Vocabulary analysis on the training split. Left: cumulative coverage and UNK rate as a function of K on a log scale, with the configured pipeline cap and 80%/90%/95% reference points marked. Middle: float32 embedding-table size in MB versus K for embedding dimensions $d \in \{64, 128, 256\}$. Right: the long-tail breakdown of SKU frequency, contrasting the share of SKUs against the share of events they account for.

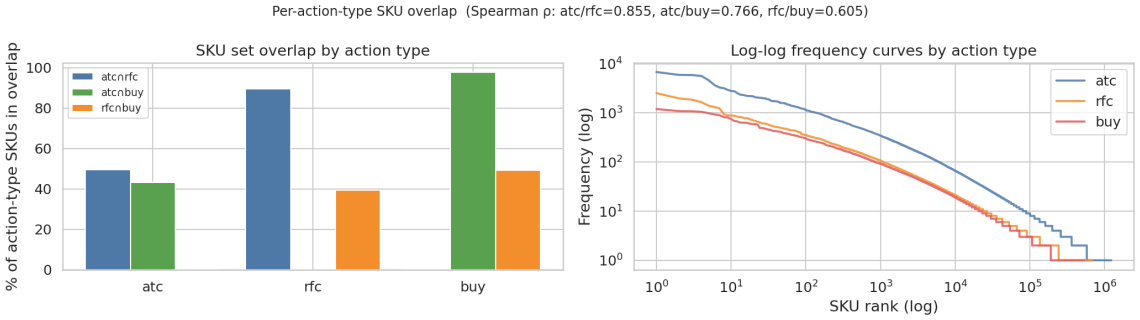

Fig. 3. Per-action-type SKU distributions on the training split. Left: pairwise SKU set overlaps. Right: log–log frequency curves for `add_to_cart`, `remove_from_cart`, and `product_buy`, all showing the same approximate power-law decay.

The interaction stream is not only long-tailed at the item level; it is also structured at the session level. Following a priority-based labelling (`purchase` ≻ `cart` ≻ `search` ≻ `browse`, where each session is labelled by the highest-priority event it contains), the training split decomposes into 2.0% pure browse sessions, 32.8% search-only sessions, 45.2% sessions that reach the cart but not checkout, and 19.9% purchase sessions. Figure 4 visualizes this distribution alongside

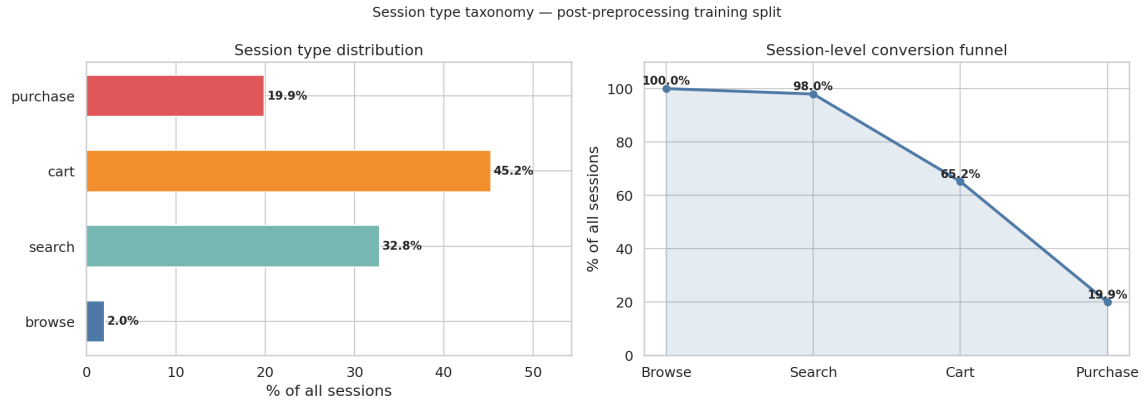

Fig. 4. Session type taxonomy and conversion funnel on the training split. Sessions are labelled by their highest-priority event. The funnel makes explicit that purchase-bearing sessions are a one-in-five minority that any session generator must reproduce in the right proportion rather than collapse into the dominant browse/search/cart modes.

the implied session-level conversion funnel (100% → 98.0% → 65.2% → 19.9%), and 26.2% of training users complete at least one purchase.

The dataset is well-suited for evaluating interaction generation because it contains temporally ordered event stream, as well as a domain-specific behavioral funnel spanning browsing, search, cart management, and purchase. All the while, the constraints extracted in this thesis are retail-specific. The resulting validity rules, exposure prerequisites, and validity checks are therefore conventions of e-commerce interaction data, not as domain-agnostic behavioral rules that would automatically transfer to settings such as media, news, or social feeds.

5.2 Preprocessing Pipeline

The preprocessing pipeline is implemented in `preprocess.py` and `ingestion/dataset.py` , and it consists of six main stages.

(1) **Schema normalization.** The five raw event parquet files are merged into a unified interaction table with a shared schema over `client_id`, `timestamp`, `event_type`, and `sku`. Product metadata is then joined onto item-bearing events, adding category and price information from `product_properties`. This provides a consistent event stream for both session construction and downstream feature extraction.

(2) **Deduplication and timestamp cleanup.** Exact duplicate item-bearing events are removed when they share the same `client_id`, timestamp, and SKU. Interactions with the same user and timestamp but different SKUs are retained, since they plausibly correspond to multi-item basket actions or simultaneous checkout events. This step removes 161,673 exact duplicates from `add_to_cart`, 153,675 from `remove_from_cart`, and 380,923 from `product_buy`. Timestamps are also checked against the dataset bounds (23 June 2022 to 8 December 2022).

(3) **Sessionization.** User timelines are segmented with a 30-minute inactivity threshold[13]. This choice was decided based on the observed gap distribution. Figure 6 shows the distribution of all within-user inter-event gaps in the post-preprocessing corpus, along with the CDF zoomed into the 0-60-minute window where any reasonable sessionization threshold must sit. The CDF is essentially flat in the 20 to 45-minute range, so the threshold's exact value has only a minor effect on the fraction of gaps it cuts. Table 3 quantifies this, varying the

threshold across $\{20, 30, 45\}$ minutes changes the number of sessions by less than 5.5% while preserving median and tail percentiles. Figure 7 shows the same sensitivity analysis visually, together with the session-length CDF and the bucketed length distribution under the chosen 30-minute threshold. Sessions containing only `page_visit` events are removed afterward because they provide no item signal and zero of such sessions remain in the final preprocessed corpus.

(4) **Time-aware split by session start.** Train/validation/test assignment is determined by session start time rather than individual event time, and rather than by user or by random shuffle. The session-start choice avoids in-session leakage[26]: a per-event time split would place events of the same session on both sides of the boundary, leaking the session-prefix signal that the generator is supposed to learn. The time-based choice is preferred over a user-based or random split because the deployment regime of the simulator is forward-in-time, the generator is expected to produce sessions for the period after the training window, not for held-out users sampled uniformly from the same period, and because a random event-level shuffle would leak both temporal and intra-session structure simultaneously. Sessions starting before 15 November 2022 are assigned to training, sessions starting from 15 November to 1 December 2022 to validation, and sessions starting on or after 1 December 2022 to test. The cutoffs partition the 168-day observation window into roughly 145/16/8 days and place the densest, behaviorally distinct late-season traffic (the Black Friday and early holiday-shopping period) on the held-out side, so the validation and test splits function as a forward-in-time test. The split itself is deterministic given the configured cutoffs and involves no randomness, so it contributes nothing to the seed-to-seed variance reported in Section 7.1. The realized split sizes are reported in Table 4.

(5) **Vocabulary capping.** The item vocabulary is truncated to the most frequent SKUs in the training split. The configured value is $K = 200{,}000$, which corresponds to 199,999 explicit SKU entries plus a shared zero index used for padding, non-item events, and out-of-vocabulary items. Under the realized session-start training split, this vocabulary covers 71.71% of training item-bearing events, a training OOV rate of 28.29%. Reaching 90% coverage would require 654,304 explicit SKUs and 95% would require 887,771 (see Table 2 and Figure 2). Relative to the train top-K vocabulary, exact event-level OOV rises substantially on the held-out splits, as reported in Table 5, illustrating strong temporal item drift.

(6) **Constraint rule extraction.**
Valid action transitions and exposure-style item prerequisites are extracted from the training split. Figure 5 shows the full row-normalised action bigram matrix together with the raw transition counts on a logarithmic colour scale. All 25 action pairs are observed at least once, but six pairs sit below a 1% row-probability threshold and are therefore candidates for soft rather than hard transition constraints. The two least likely transitions are `search_query` → `product_buy` (p=0.0007) and `search_query` → `remove_from_cart` (p=0.0016); the rarest transition originating from a purchase is `product_buy` → `remove_from_cart` (p=0.0045).
Exposure analysis is summarized in Figure 8

Finally, the held-out splits are meaningfully harder. Figure 9 summarizes this on two axes. The left panel shows event counts per split per action type, confirming that the action mix is preserved across splits; the right panel shows the user-overlap analysis.

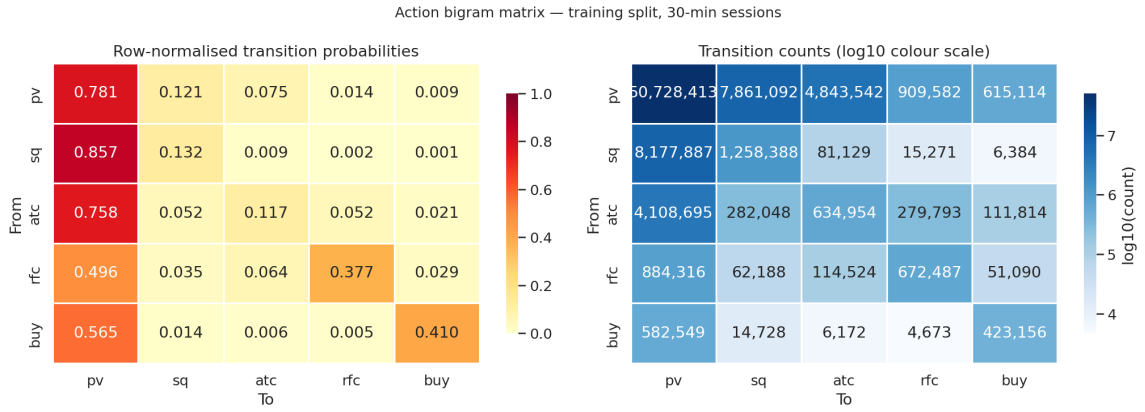

Fig. 5. Action bigram structure on the training split (30-minute sessions). Left: row-normalised transition probabilities; rows sum to one. Right: the same matrix as raw counts on a $\log_{10}$ colour scale. All 25 bigrams are observed, but six fall below the 1% row-probability threshold and are treated as soft rather than hard constraints in the generation pipeline.

Table 3. Sessionization sensitivity on the post-preprocessing corpus. The 30-minute threshold is the chosen pipeline default; baseline percentages are relative to it.

Threshold	Sessions	vs. baseline	Mean length	p90	p99
20 min	7,172,520	+5.47%	15.14	34	106
30 min	6,800,479	(baseline)	15.97	35	112
45 min	6,688,705	−1.64%	16.24	36	115

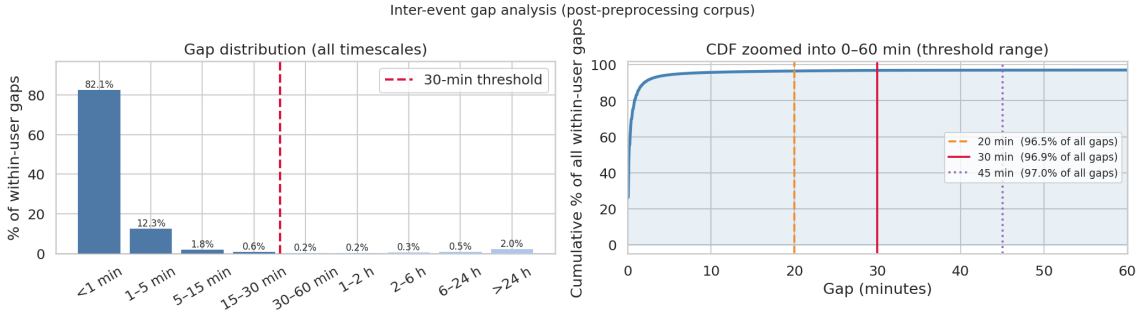

Fig. 6. Within-user inter-event gap distribution on the post-preprocessing corpus. Left: gap distribution at all timescales, with the 30-minute sessionization threshold marked. Right: CDF zoomed into 0–60 minutes; vertical lines mark the candidate thresholds $\{20, 30, 45\}$ minutes used in the sensitivity analysis.

5.3 Limitations and Trade-offs

Using an existing public dataset trades reproducibility and scale for control over schema, granularity, and sampling. The advantages are direct: 225 million events over 168 days at the session level would be infeasible to collect within an MSc timeline, and a public benchmark allows future work to reproduce the preprocessing pipeline against the same

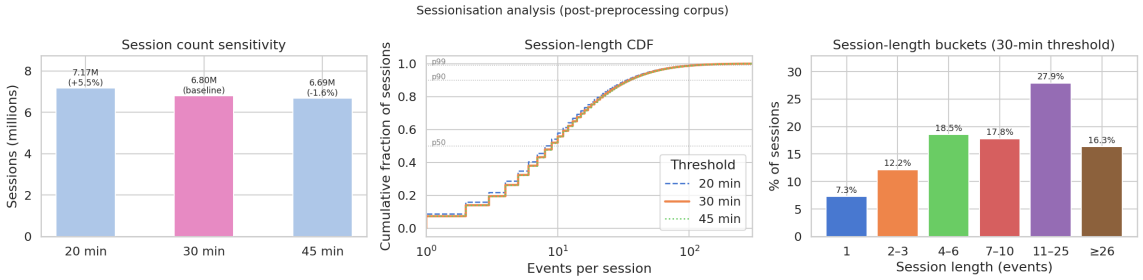

Fig. 7. Sessionization sensitivity. Left: total session count for the three candidate thresholds. Middle: session-length CDF, log-scaled on events per session. Right: bucketed session-length distribution under the chosen 30-minute threshold.

Table 4. Realized split sizes after preprocessing, page-visit-only session removal, and session-start-based splitting.

Split	Events	Sessions	Avg. session length
Train	88,170,139	5,440,150	16.21
Validation	13,689,549	910,386	15.04
Test	6,757,989	449,943	15.02
Total	108,617,677	6,800,479	15.97

Table 5. Event-level OOV rate per split under the train top-K item vocabulary ($K = 200{,}000$). The OOV rate roughly doubles from train to test, indicating strong temporal item drift.

Split	OOV rate (item-bearing events)
Train	28.29%
Validation	42.90%
Test	50.01%

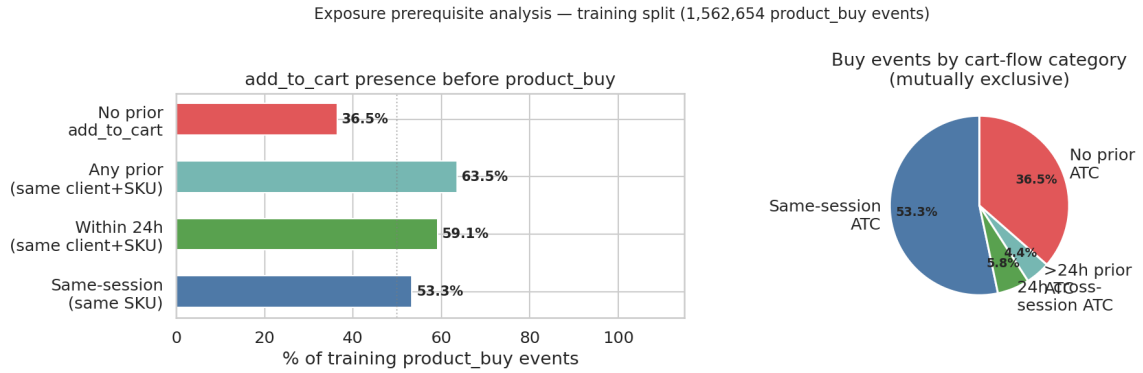

Fig. 8. Exposure prerequisite analysis on the training split. Left: share of `product_buy` events that have a prior `add_to_cart` of the same SKU, under increasingly relaxed scopes (same session, same client within 24 h, any prior). Right: the same purchases broken down into mutually exclusive cart-flow categories. Roughly one third of purchases have no prior same-SKU cart event at all.

source artefacts. The accompanying limitations are equally direct, and several are quantitative consequences of analyses already presented above rather than additional unknowns.

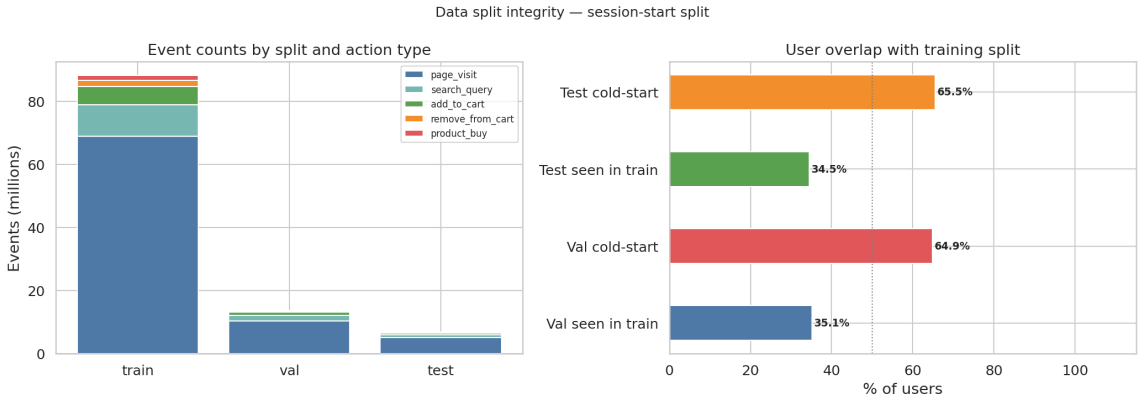

Fig. 9. Data split integrity under the session-start split. Left: stacked event counts per split, broken down by action type. Right: user overlap with the training split. Roughly two-thirds of validation and test users have no training history and are cold-start.

Vocabulary truncation is lossy. The chosen cap K=200,000 covers 71.7% of training item-bearing events (Table 2, Figure 2). The remaining 28.3% of events are mapped to the shared zero index together with padding and non-item events. Reaching 90% training coverage would require ∼650k explicit SKUs, and reaching 95% would require ∼890k. The pipeline therefore explicitly chooses to under-represent the long tail in exchange for a tractable embedding table and lower training-time costs, results in the tail are not directly addressable by the generator.

Strong temporal item drift. Relative to the train top-K vocabulary, event-level OOV nearly doubles from training to test (Table 5). This reflects the catalog itself shifting over the held-out window rather than only sampling noise, so absolute test metrics conflate generation quality with item-vocabulary drift. This is mentioned qualitatively by the K cap discussion above and by reporting both train- and test-vocabulary OOV, but it cannot be eliminated without re-training on a moving window.

Cold-start dominates the held-out splits. Approximately two-thirds of validation and test users do not appear in training (Figure 9). User-conditioned generation is therefore evaluated mostly on users for whom no historical context exists. This means that personalization metrics are upper-bounded by population-level priors for the majority of held-out users.

Soft rather than hard behavioral constraints. Exposure analysis (Figure 8) shows that only 53.3% of training purchases follow a same-session same-SKU `add_to_cart`, and 36.5% have no prior same-SKU cart event of any kind. A hard prerequisite would therefore mark over a third of valid purchases as invalid. The pipeline accepts the resulting modeling complexity by combining hard validity checks (the bigram structure of Figure 5) with softer behavioral constraints, rather than enforcing a single rigid rule.

Item names are opaque numeric vectors, not text. The `name` field is published as a 16-byte numeric vector rather than a readable string. The bytes are near-uniformly distributed (99.9% of maximal entropy) yet carry real category structure (nearest-neighbour same-category rate 19.71% versus 0.25% frequency-weighted chance, a 78.85× lift), so the field is a quantized pre-computed embedding rather than either random noise or tokenizable text. The thesis therefore uses `name`

as a learned input feature embedding (Section 6.3), but cannot deploy text-encoder-based item heads such as two-tower retrieval over a text encoder, name-conditioned zero-shot generation, or BM25 candidate generation.

Domain and provenance are retail-specific. The dataset is a single-vendor (Synerise) retail e-commerce log released for the RecSys 2025 challenge. Three caveats follow. First, the legality rules, exposure prerequisites, and validity checks extracted here are conventions of e-commerce interaction data and do not transfer automatically to settings such as media, news, or social feeds. Second, the data is collected from one tracking platform serving a particular set of merchants and audiences; demographic, geographic, and device-mix biases are present but not directly observable in the schema. Third, anonymization is provider-supplied: `client_id` and `sku` appear as opaque integers, and no claim of k-anonymity or differential-privacy guarantees is made by the release. The thesis treats these identifiers as opaque keys and does not attempt re-identification.

5.4 Data Handling and Governance

The dataset consists of anonymized behavioral logs released for the RecSys 2025 challenge. No personally identifiable information is processed: `client_id` and `sku` are opaque integer identifiers, and no free-text user content, contact information, or precise geolocation is present in the schema. The legal basis for use is the dataset's published challenge licence, which is documented in Section 4.

Anonymization is supplied by the data provider rather than introduced as part of this thesis, and the release does not state a formal k-anonymity or differential-privacy guarantee. The thesis therefore treats the identifiers as opaque and makes no attempt to re-identify users, link the data to external sources, or invert the anonymization. All analyses, models, and metrics reported in the thesis are population-level; no individual-level records, sequences, or generated samples that could be traced to a real `client_id` are published.

Storage is restricted to the project's working environment for the duration of the thesis. Intermediate parquet files, learned vocabularies, and model checkpoints are kept on local project storage and are not redistributed. Aggregated statistics and figures published in the thesis carry negligible re-identification risk because they are computed over millions of events and hundreds of thousands of users; no cell or bin in any reported table or figure is small enough to single out an individual user or SKU.

6 Methodology: Model and Architecture

6.1 Interaction Schema and Interface Contract

The minimum generated event tuple is defined in Table 6. This schema formalizes what "interaction" means in the simulation context and determines what each prediction head of the generator must produce.

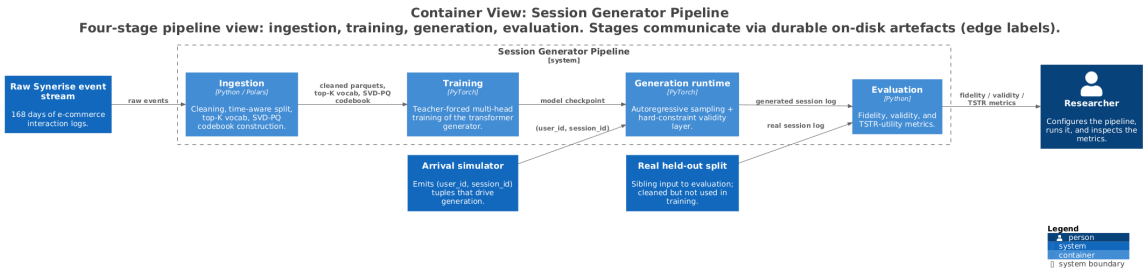

Fig. 10. Container-level view of the system (C4 container diagram) [4] Stages communicate through durable artefacts (edge labels) rather than in-memory handoffs, so any single stage can be re-run with its inputs held fixed. The model internals are detailed in Section 6.3; the generation-time component view is given in Figure 12 (Appendix A).

Table 6. Output log schema. Fields marked *generated* are produced directly by the generator; *derived* fields are computed post hoc.

Field	Type	Description
`user_id`	integer	*Generated by arrival process.* Identifies the synthetic user.
`session_id`	integer	*Generated by arrival process.* Groups events into one session.
`action_type`	categorical	*Generated by transformer model.* One of: `page_visit`, `search_query`, `add_to_cart`, `remove_from_cart`, `product_buy`.
`item_id`	integer	*Generated by transformer model.* The item involved in the event. Null for `search_query` events, which do not target a specific item.
`delta_t`	float	*Generated by transformer model.* Inter-event time in seconds.
`event_time`	timestamp	*Derived.* Cumulative `delta_t` from session start.

The `item_id` field is nullable; the item prediction head is suppressed for `search_query` and `page_visit` events and no item target is produced or evaluated at those positions, as these action types do not contain item-information in the dataset. Session boundaries are determined by an EOS token predicted by the generator, subject to a maximum session length cap. The generator accepts the output of the arrival simulator as input and produces the schema in Table 6. Context features such as category and price are excluded from the output schema, but are included as input for the model.

6.2 System Architecture

The pipeline is organized into four stages: *ingestion*, *training*, *generation*, and *evaluation*. Each stage runs as an independent process, and stages communicate exclusively through durable artefacts on disk rather than through in-memory handoffs. .Any single stage can be re-run while the upstream artefacts are held fixed. Figure 10 gives the container-level view of the four stages and the artefacts that connect them. This subsection then walks through each stage in turn, deferring the model internals to Section 6.3 and the validity layer to Section 6.4. This four-stage decomposition extends the simulator architecture of supervisor Diptish Dey from the CMI lab.

Stage 1: Ingestion. The raw Synerise event stream is cleaned, time-aware split into train, validation, and test partitions, and reduced to a top-K item vocabulary that maps the long catalog tail to a shared zero index. Offline, a second ingestion job constructs the SVD-PQ codebook [28] used by the third model iteration: a truncated SVD of the weighted user-item

interaction matrix followed by per-dimension quantile binning, producing one fixed-length token tuple per SKU. The full preprocessing pipeline, the rationale for K, and the empirical justifications for the split and the codebook threshold are detailed in Section 5 and Section 6.3.3.

Stage 2: Training. The dataset loader reads the cleaned parquet partitions and emits (prefix, target) pairs together with a cross-session memory window of the user's H most recent prior events. The transformer is trained with teacher forcing and a summed cross-entropy loss over the three factored heads (action, item, inter-event time)[34]; the EOS class on the action head is the final target of every training session, so session termination is learned from real boundaries rather than imposed by a fixed cap. The output of the stage is a single model checkpoint. The head architecture itself, including the three iterations of the item head, is described in Section 6.3; the search procedure and final hyperparameter values are in Section 6.6.

Stage 3: Generation runtime. The arrival simulator yields (`user_id`, `session_id`) tuples that drive a session-based autoregressive generator. For each session, the trained checkpoint is unrolled step by step, with the input embedding summing the running session prefix and a cross-session memory window over the same user's prior sessions; at each step the three factored heads jointly produce an action, an item, and an inter-event time. The session terminates on EOS. Before the session is written to the output log it passes through the hard-constraint validity layer (Section 6.4), which is reported both pre- and post-repair so that the cost of constraint enforcement is independently measurable. Figure 12 in Appendix A shows the component-level view of this stage.

Stage 4: Evaluation. The evaluation stage consumes the generated session log and the real held-out parquet split as sibling inputs and produces the fidelity, validity, and downstream-utility (TSTR) metrics defined in Section 7. Because the inputs to this stage are plain logs that share the schema of Table 6, the same evaluation harness is used for the transformer generator and for the Markov baseline of Section 6.5, which is the prerequisite for treating the comparison as a fair test.

Interfaces and artefacts. The artefacts that pin the system down are the cleaned and split parquets, the top-K vocabulary file, the SVD-PQ codebook[28], the trained model checkpoint, and the generated session log (whose schema is Table 6). Each artefact has a stable on-disk representation, so the four stages can be developed and re-run independently as long as the upstream artefact is fixed.

6.3 Model Architecture

The project evaluates three model architectures across successive iterations. All three share the same transformer backbone, action head, temporal head, and cross-session memory and differ only in how the next SKU is predicted from the shared hidden state. The common scaffold is described once below: each item-prediction variant is then summarized in turn. The corresponding architecture diagrams are reproduced in Appendix A.

Shared scaffold. At every position the input representation is the sum of action, item, inter-event-time, positional, and auxiliary item-side embeddings (category, price, name). A causal transformer decoder[33] attends over the running session prefix together with a cross-attention memory built from the user's prior sessions (window of H events) and emits a hidden state at each step. Three factored heads share this hidden state: an action head over the five action types plus an end-of-session class, an item head whose form is the subject of the iterations below, and a temporal head over

Table 7. Cross-session memory window coverage. "Lossless coverage" is the share of training users whose entire event history fits inside the window; cross-attention cost scales as $O(L \cdot H)$ per layer relative to $H = 16$.

H	Lossless coverage	Cross-attn cost (rel.)
16	60.9%	1×
32	79.2%	2×
64	90.5%	4×
128	96.2%	8×
256	98.7%	16×

discretized log-spaced inter-event-time bins. Training is teacher-forced with summed cross-entropy losses on the three heads (`simulation/generator/session_transformer.py`).

Cross-session memory window. The window size $H = 16$ is chosen to match the mean session length on the cleaned training corpus ($\overline{|s|} = 15.97$ events), so that on average one full prior session of context is available at each generation step. This sets the cross-attention budget at the natural unit of behaviour the generator operates on a session, rather than at an event count. Under this choice the entire prior event history of 60.9% of training users fits losslessly inside the window, while the remaining users contribute their most recent H events. The diminishing returns of larger H are summarized in Table 7: cross-attention cost doubles each time H doubles, while the marginal lossless-coverage gain shrinks. $H = 16$ therefore sits on the cheap side of the coverage–cost knee, with $H = 32$ identified as the natural ablation should long-history users prove undersampled.

6.3.1 Iteration 1: Flat softmax over the full SKU vocabulary. Concept. A vanilla baseline that predicts the SKU directly from the hidden state with a single K-way softmax. The value proposition is methodological: before adding any structural prior to the item head, establish whether the simple formulation already produces acceptable fidelity, validity, and bias readings, so that subsequent iterations have a real baseline to improve upon rather than a strawman. *Test setup.* Trained on the train split and evaluated on the validation split using the fidelity, validity, and bias metrics of Section 7. As the first iteration, its readings serve as the reference point that informs the design choices of Iteration 2 rather than as a pass/fail signal.

The baseline iteration predicts the next SKU with a single softmax over the entire top-K vocabulary ($K \approx 200{,}000$). The item head (`ItemHead` in `heads.py` with `n_categories=0`) is a linear projection followed by a K-way softmax, conditioned only on the sampled action. Training is vanilla cross-entropy over the global SKU index, with no masking, no factorization, and no offline preprocessing.

The flat formulation has two known weaknesses that motivate the later iterations. The softmax denominator runs over all K classes at every item-bearing position, so the gradient signal on tail SKUs is diluted by competition with the popularity head; and the head treats SKUs as independent symbols, with no built-in notion of which items are substitutable.

6.3.2 Iteration 2: Hierarchical category → item. Concept. Introduce the dataset's category labels as a hard structural prior on the item head, so that the popularity-driven gradient signal of Iteration 1 is broken into a category-level decision followed by a much smaller in-category decision. The value proposition is targeted towards the bias group (item coverage, popularity JSD, ΔGini), which was the failure mode of Iteration 1, and category-level factorization addresses it without changing the action or temporal heads. *Test setup.* Trained on the train split and evaluated on the

validation split against the Iteration 1 reference point on the same metric suite. Pass condition: improved item coverage and reduced ΔGini relative to Iteration 1 without regressing the action and temporal fidelity metrics.

The second iteration exploits the dataset's category labels as a hard structural prior. A new `CategoryHead` predicts a category index from the hidden state, and the item head then samples within the SKU pool belonging to that category. At training time this is realized as a vectorized masked cross-entropy: full K-way logits are computed, out-of-category positions are set to $-\infty$, and standard cross-entropy is applied (`ItemHead.hierarchical_loss`). At inference the same masking is realized by slicing the head's weight matrix to the rows of the sampled category and renormalizing (`_sample_flat_pool`). Empty-pool categories are masked out of the category softmax to avoid emitting padding events.

The intent is to redistribute probability mass. Instead of a single K-way softmax that the popularity head can dominate, the model now spends a category-level softmax to pick a category and a much smaller in-pool softmax (typically two to three orders of magnitude smaller than K) to pick the SKU. Tail items only compete against their category siblings, which directly attacks the coverage collapse of Iteration 1. The architecture diagram is in Figure 14.

6.3.3 Iteration 3: SVD-PQ token-factored item head. Concept. Replace the per-SKU softmax with a small set of shared discrete tokens drawn from an offline SVD-PQ codebook [18, 28], so that semantically related SKUs share leading tokens and therefore share gradient signal. The value proposition is twofold: it spreads the item-side gradient across tv token cells rather than K output classes (so tail SKUs are no longer drowned out at training time even within a category pool) and it collapses the head's parameter count from $O(Kd)$ to $O(tvd)$, freeing VRAM that can be redirected into a deeper backbone or longer cross-session memory. *Test setup.* Trained on the train split and evaluated on the validation split against Iteration 2 on the same metric suite, augmented with the in-training TSTR probe of Section 7.4 as the model-selection signal. Pass condition: bias group on par with or better than Iteration 2 *and* TSTR HR@10 above Iteration 2 on the validation split, so that the gain is not only distributional but also actually helpful for downstream recommender training.

Each SKU is encoded offline as a fixed-length tuple of t discrete tokens drawn from v bins, derived by truncated SVD [28] of the weighted user–item interaction matrix followed by per-dimension quantile binning (`ingestion/svdpq.py`). The project uses $t = 4$ tokens of $v = 512$ bins. Items that interact with similar users land in nearby quantization cells, so semantically related SKUs share leading tokens.

Event-type weighting. Each (*user*, *item*) cell of the interaction matrix is the sum of per-event weights, where the weight assigned to an event depends only on its action type. The weights follow an inverse-frequency scheme: for each action type a with corpus count n_a in a corpus of N events, the weight is $w_a = \log(N/n_a)$. Table 8 reports the resulting values on the cleaned event corpus. The intent is to upweight rare but informative actions (purchases, cart events) over the abundant but noisy page-visit signal, so the SVD factors are dominated by events that actually express preference rather than by browse traffic.

The item head (`SVDPQItemHead` in `heads.py`) is a single linear projection whose output is reshaped into t parallel v-way logit vectors. The training loss is the mean cross-entropy over those t token positions, with optional per-dimension label smoothing to bound the joint peak probability.

This collapses the head's parameter count from order $K{\cdot}d$ to order $tv{\cdot}d$ ($\approx 200{,}000\,d \rightarrow 2{,}048\,d$ in this setting), and the training-time softmax denominator runs over $tv = 2{,}048$ outputs rather than K. Following GPTRec[28], forcing related SKUs to share leading tokens is expected to spread the gradient signal across token cells rather than concentrating it on individual head SKUs, mitigating the mode-collapse pathology of Iteration 1. The architecture diagram is in Figure 15.

Table 8. Per-action SVD-PQ event weights $w_a = \log(N/n_a)$ computed on the cleaned event corpus ($N = 101{,}859{,}688$ events exluding test-set).

Action type a	n_a	w_a
page_visit	79,222,116	0.25
search_query	11,649,993	2.17
add_to_cart	6,855,719	2.70
remove_from_cart	2,325,119	3.78
product_buy	1,806,741	4.03

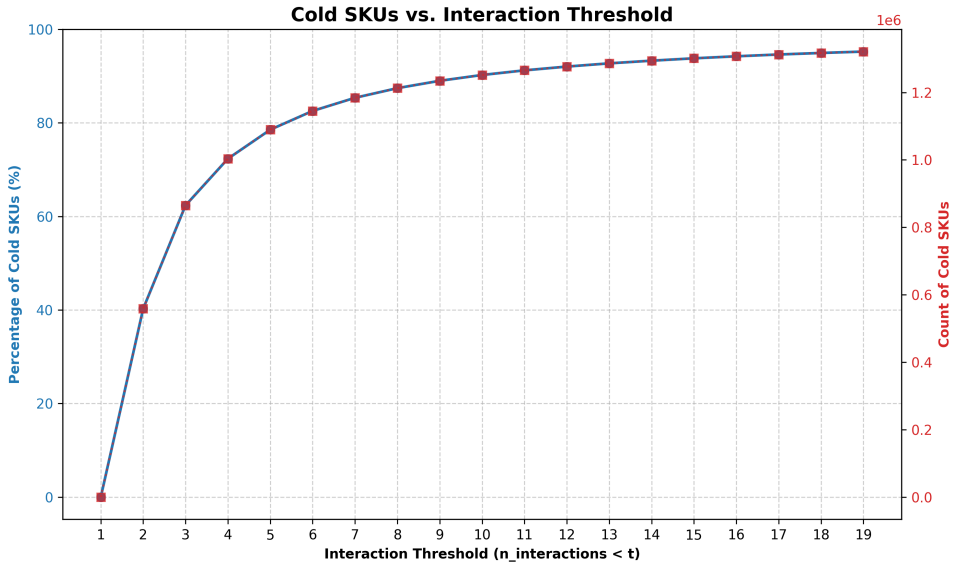

Fig. 11. Share (left axis) and absolute count (right axis) of SKUs classified as cold under varying interaction thresholds $t \in \{1, \ldots, 19\}$ (cold $\Leftrightarrow$ n_interactions $< t$). The pipeline uses $t = 2$, the smallest value at which every SVD-embedded SKU has support from at least two distinct user interactions. The curve is heavy-tailed: roughly 40% of SKUs appear once and would be unembeddable by SVD regardless of threshold choice.

Cold-SKU fallback. SKUs with too few interactions to support a meaningful SVD position are treated as cold and assigned their category centroid in token space rather than receiving an individually estimated embedding. The threshold is svdpq_min_interactions = 2: a SKU observed in only a single user–item cell yields no co-occurrence signal for SVD, so its factor-space coordinates would be fully determined by that one user's row and would not carry the relational structure the codebook is meant to capture. Two is therefore the minimum value at which every SVD-embedded SKU appears in at least two distinct user contexts. Figure 11 shows the trade-off: at this threshold, roughly 40% of catalog SKUs fall back to the category centroid, but these are by construction the rarest SKUs and account for a correspondingly tiny share of training item-bearing events. Raising the threshold further (e.g., to 5) would push $\approx 78\%$ of SKUs into the fallback while discarding usable co-occurrence signal from items that do have multi-user support.

6.4 Hard-Constraint Validity Layer

Generated sessions pass through explicit validity checks before output:

(1) **Invalid action transitions:** action bigrams not observed in training data are flagged and counted.
(2) **Timestamp monotonicity:** events where $\delta_t < 0$ are flagged.
(3) **Purchase-without-prior-exposure:** a `product_buy` event with no preceding `page_visit` or `add_to_cart` within the session is flagged.

Both pre-constraint and post-constraint statistics are reported, making the cost of constraint repair independently measurable.

Design safeguards. The validity layer is an AI-specific guardrail in the generation pipeline against behaviorally implausible outputs, and pre/post-repair reporting turns its decisions into an auditable signal rather than a silent rewrite. The cold-SKU fallback (Section 6.3.3) complements it as a *graceful-degradation* path for SKUs with insufficient co-occurrence signal, avoiding silent embedding-quality cliffs at the long tail.

6.5 Comparison Portfolio

The portfolio compares two synthetic-data generators against a real-data reference, using a fixed downstream evaluator to isolate the upstream generator as the variable of interest. The setup follows the Train-Synthetic-Test-Real (TSTR) protocol of Esteban et al. [9]: a downstream model is trained on each candidate generator's output and evaluated on a real held-out test split, on the principle that synthetic data is useful to the extent that a model trained on it transfers to real data. TSTR is therefore not a measure of distributional fidelity (that is the job of §7.2) but a proxy for downstream utility: "is this synthetic log a viable training corpus?"

Compared generators. Two generators produce the synthetic training corpora:

- **Markov generator (TSTR-M).** A first-order Markov chain over action types, with item selection drawn independently from the empirical popularity distribution of the training split. The Markov chain has no action–item coherence by construction: the sampled item does not condition on the sampled action at the same step. This generator mirrors the structure of the interaction generator currently in use within the CMI simulator (§2.4), so the comparison is directly relevant to the improvement objective of this thesis rather than against an arbitrary literature baseline.
- **Transformer generator (TSTR-T).** The proposed multi-head autoregressive transformer described in §6.3, whose factored heads emit action, item, and inter-event time jointly and are therefore coherent by construction.

Reference point. A third condition, **TRTR** (Train on Real, Test on Real), trains the same downstream evaluator on the real training split. TRTR is not a generator and is not in the head-to-head comparison; it is an interpretive ceiling that establishes what the downstream evaluator achieves with no generator in the loop. A TSTR score approaching TRTR indicates that the generator's output preserves the downstream-relevant structure of the real data; a TSTR score exceeding TRTR is a diagnostic signal that the synthetic data presents an artificially easy task (see §8.3).

What is held constant. The downstream evaluator is GRU4Rec [14], trained with identical architecture, hyperparameters, optimizer settings, and early-stopping criteria across TSTR-M, TSTR-T, and TRTR. Only the training data source differs. Any difference in HR@10 or NDCG@10 between TSTR-M and TSTR-T is therefore attributable to the upstream generator rather than to downstream modeling choices. The test split is identical across all three conditions, so the evaluation target is also held fixed.

Scope of the resulting claim. The claim supported by this portfolio is narrow and should not be overstated. A favorable TSTR-T result, if obtained, shows that *a constraint-aware multi-head transformer outperforms a popularity-sampled first-order Markov baseline on the Synerise dataset under this evaluation protocol, for the purpose of producing training data for GRU4Rec*. It is not a general claim that transformer architectures dominate Markov-chain generators for synthetic interaction data; that would require a broader baseline portfolio including GRU- or GAN-based generators and replication across datasets, both of which are scoped to future work (§9). The portfolio is sized to support a defensible improvement claim against the specific baseline this thesis is positioned to replace, not to settle an architectural debate.

6.6 Hyperparameter Tuning and Training Procedure

The pipeline is configuration-driven: every tunable knob is exposed through a single YAML file (`config.yaml`) and consumed by the ingestion, training, generation, and evaluation stages without code changes. Table 9 enumerates the hyperparameters together with the configuration key under which each is set; search ranges and the strategy used to select final values are reported below the table.

[To write: search range per knob, selected final values, total wall-clock training time.]

7 Methodology: Evaluation

7.1 Evaluation Setup

Evaluation covers three dimensions: fidelity (do generated sessions look like real ones?), validity (do they respect behavioral constraints?), and downstream utility (do they train a better recommender?). All metrics are reported across five random seeds as mean and standard deviation. Reference distributions for the fidelity metrics are precomputed once from the training split via a `ReferenceStore`, so the synthetic seeds are scored against a fixed target rather than a per-seed re-profile. The evaluation design is grounded in the reliability and validity criteria in Section 7.5.

7.2 Fidelity Metrics

The fidelity layer was redesigned to probe distinct failure modes observed in early iterations rather than report a single distance. Each metric targets a separable property of the generated log; a generator can pass one and fail another, and the combination is what supports a fidelity claim. The full set is summarized in Table 14 (Appendix B).

Five distributional metrics compare the generated sessions against the real training distribution, two behavioral-rate metrics check funnel mechanics, and a third group of bias metrics is reported to track item-side mode collapse without entering the primary fidelity claim[15, 20]. All bias metrics are computed against a size-matched random subsample of the real split (matched on session count to the synthetic batch); without this, item coverage and Gini are mechanically depressed for any synthetic batch smaller than the multi-million-session reference.

The L1 distance on the item bigram transition matrix was considered and discarded after inspection: the bigram matrix is so sparse that the metric is dominated by which UNK-adjacent transitions either side happen to sample, and it adds no signal beyond the popularity JSD and item-coverage bias metrics that already cover item-side concentration.

Both generators are evaluated on the identical metric set, so the TSTR-T versus TSTR-M comparison is direct.

7.3 Validity Metrics

Generated sessions are checked against three hard behavioral constraints extracted from the training data (`evaluation/validity_checker.py`):

Table 9. Tunable hyperparameters of the pipeline, grouped by stage. Configuration keys correspond verbatim to entries in `config.yaml`. Search ranges and the selected values are reported below the table.

Group	Config key	Description
Sessionization	`session_timeout_min`	Inactivity gap that starts a new session (min).
	`history_window`	Cross-session memory window H (events).
	`train_cutoff`	Train/validation split date (session-start).
	`val_cutoff`	Validation/test split date (session-start).
Vocabulary	`vocab_k`	Top-K item vocabulary cap.
Temporal head	`n_temporal_bins`	Number of log-spaced δ_t bins.
	`temporal_min_s`	Minimum representable inter-event delta (s).
Transformer	`train_d_model`	Hidden dimension d.
	`train_n_layers`	Number of decoder layers L.
	`train_n_heads`	Attention heads (must divide d).
	`train_max_length`	Max events per session, including EOS.
	`train_batch_size`	Mini-batch size.
	`train_lr`	AdamW learning rate.
	`train_epochs`	Maximum training epochs.
	`train_patience`	Probes without HR@k gain before stop.
Category head	`category_rare_threshold`	SKU count below which a category is collapsed to RARE.
SVD-PQ item head	`svdpq_t`	Tokens per SKU.
	`svdpq_v`	Bins per token dimension.
	`svdpq_label_smoothing`	Per-dim CE label smoothing.
	`svdpq_binning`	`quantile` or `uniform`.
	`svdpq_noise_std`	Pre-bin Gaussian noise stddev.
	`svdpq_min_interactions`	Cold-SKU fallback threshold.
Loss weighting	`loss_weights_enabled`	Enable per-head loss weighting.
	`loss_weights`	4-entry vector: action, category, item, temporal.
Bias mitigation	`item_loss_freq_weight_alpha`	Inverse-frequency item-CE exponent α.
	`cat_logit_adj_tau`	Menon logit-adjustment strength τ.
Inference	`infer_temperature`	Action / category / temporal softmax temperature.
	`infer_item_temperature`	Item-head sampling temperature.
	`infer_pool_temperature`	SVD-PQ in-pool softmax temperature.
	`svdpq_infer_scorer`	`hamming` or `log_prob` pool scorer.
In-training TSTR probe	`is_best_eval_enabled`	Use TSTR HR@k as best-ckpt signal.
	`is_best_eval_every_n_epochs`	Probe cadence (epochs).
	`is_best_eval_n_sessions`	Synthetic sessions per probe seed.
	`is_best_eval_k`	Cutoff k for HR@k / NDCG@k.
Final evaluation	`final_eval_n_sessions`	Synthetic sessions per evaluation seed.
	`final_eval_seeds`	Random seeds for multi-seed reporting.
GRU4Rec downstream	`gru4rec_embed_dim`	Item embedding size.
	`gru4rec_hidden_dim`	GRU hidden size.
	`gru4rec_max_epochs`	Maximum training epochs.
	`gru4rec_batch_size`	Mini-batch size.
	`gru4rec_lr`	Adam learning rate.
	`gru4rec_patience`	Early-stopping patience.

- **Invalid action transition rate** — fraction of sessions containing at least one action bigram not observed in the training data.

- **Timestamp monotonicity violation rate** — fraction of sessions with any $\delta_t \leq 0$.
- **Purchase-without-prior-exposure rate** — fraction of sessions in which a `product_buy` occurs on an SKU that has not appeared in a prior `page_visit` or `add_to_cart` within the same session.

Violation rates are reported both before and after the hard-constraint validity layer, making the repair cost of the constraint layer visible.

7.4 Downstream Utility (TSTR)

Utility follows the Train-Synthetic-Test-Real (TSTR) paradigm [9, 19]:

(1) Train each generator on the training split.
(2) Generate a synthetic training set of equal session count.
(3) Train GRU4Rec on the synthetic set; evaluate on the real held-out test split.
(4) Compare TSTR-T against TSTR-M on HR@10 and NDCG@10.

A Train-Real-Test-Real (TRTR) run on the same GRU4Rec recipe provides an interpretive reference point. The primary claim is directional: TSTR-T should outperform TSTR-M on both HR@10 and NDCG@10. Improvement on one metric only is reported but treated as inconclusive. HR@10 and NDCG@10 are the standard benchmarks in sequential RS research [14, 21, 31] and follow the TSTR evaluation framing of Jordon et al. [19].

7.5 Reliability and Validity

Reliability. Test–retest reliability is operationalized through five-seed variance reporting. Running the same generator with five different random seeds plays the role of repeated measurements: low variance across seeds indicates that the measurement is stable and not an artifact of random initialization. This follows the repeated-run practice recommended by T-RECS [24].

Construct validity. The fidelity metrics are non-redundant. JSD over action types captures whether the generator produces the right behavioral mix at the marginal level [22]; the action bigram L1 goes one level deeper and checks whether the sequential ordering of actions is preserved (a generator can pass JSD and fail bigram L1 by producing the right action mix in the wrong order). KS on session length and KS on δ_t separate two distinct temporal failure modes — wrong session shape versus wrong inter-event pacing — that a single aggregate would hide. The conversion-rate and cart-abandonment deltas anchor the generator to the funnel mechanics that downstream recommender quality ultimately depends on. The sample-diversity ratio guards against the case in which all distributional marginals match but every session is a near-duplicate. The bias group (coverage, popularity JSD, Gini delta) is kept separate from the primary fidelity claim because it diagnoses an item-side pathology — mode collapse onto the head — that is the explicit target of Iterations 2 and 3, and so is reported as a development signal rather than a pass/fail metric.

Ecological validity. Offline evaluation results are bound to static log fidelity and training-set utility; they do not support claims about generator performance under live deployment with real feedback dynamics.

Content validity. The five-action vocabulary covers the complete observed action space of the Synerise dataset. No action type is excluded from the evaluation.

8 Results (Preliminary)

Status note. Single-seed results for Iterations 2 and 3 only. Two caveats apply throughout:

- **Single seed.** seed = 42; the five-seed protocol of Section 7.1 has not yet been run.
- **Iteration 1 omitted.** The flat-softmax baseline was evaluated under an earlier preprocessing configuration and is being retrained against the current one as of writing.

8.1 Fidelity

Validation-split fidelity is reported in Table 10. Both transformers are co-evaluated against an identical Markov baseline and a real-vs-real TRTR reference (the latter sets the empirical floor any generator can plausibly reach). The number of generated sessions for validation were 500.000.

Table 10. Preliminary validation-split fidelity (single seed). Lower is better except where marked † (ratio metrics, 1.0 ideal).

Metric	Iter 2	Iter 3	Markov	TRTR
Distributional				
JSD(action)	0.0035	0.0034	0.00047	0.00048
KS(session length)	0.189	0.139	0.946	0.040
KS(δ_t)	0.251	0.251	0.251	0.041
L1(action bigrams)	0.154	0.128	0.054	0.055
L1(item bigrams)‡	1.787	1.894	1.821	1.500
Sample diversity†	0.999	1.000	1.000	1.000
Behavioral rate				
Δ conversion rate	0.091	0.115	0.132	0.009
Δ cart abandonment	0.086	0.088	0.239	0.024
Bias (informational)				
Item coverage†	0.258	1.539	2.275	1.714
Unique synth. SKUs	17,698	105,732	155,912	126,015
Popularity JSD	0.469	0.487	0.455	0.295
Δ Gini	0.220	0.014	0.338	0.090

‡ Sparse-bigram saturation makes this metric weakly informative on the top-K vocabulary (Section 7.2).

Markov vs. transformer split as expected. Markov is near-optimal on metrics it can hit by construction (action JSD, action bigram L1) and fails on session shape (KS = 0.946 vs. 0.139–0.189 for the transformers).

Iteration 3 improves on Iteration 2 on action and temporal fidelity. KS(session length) drops 0.189 → 0.139, action bigram L1 drops 0.154 → 0.128. The pre-registered "no regression on action/temporal" pass condition is met by an improvement margin.

The bias group is a trade-off. Iteration 2 mode-collapses (coverage 0.258, ΔGini 0.220); Iteration 3 over-diffuses (coverage 1.539, ΔGini 0.014). Popularity JSD is essentially flat across both (0.469 vs 0.487), both well above the TRTR reference of 0.295, neither head matches real popularity shape. The category prior reduces mode-collapse severity without eliminating it; the SVD-PQ factorization [28] shifts the failure mode rather than removing it.

KS(δ_t) is identical to four decimal places across both iterations and Markov (0.251).

8.2 Validity

Both iterations satisfy the action-transition and monotonicity constraints at generation time. The purchase-prior-exposure repair cost (∼14%) is essentially identical across iterations and sits below the 36.5% rate at which real training purchases lack a prior same-SKU cart event (Section 5, Figure 8); both generators are stricter about cart-before-purchase ordering than the real distribution.

Table 11. Pre- and post-repair validity rates (single seed). Lower is better.

	Pre-repair		Post-repair	
Constraint	Iter 2	Iter 3	Iter 2	Iter 3
Invalid action transition rate	0.000	0.000	0.000	0.000
Timestamp monotonicity violations	0.000	0.000	0.000	0.000
Purchase-without-prior-exposure	0.144	0.142	0.000	0.000

8.3 Downstream Utility (TSTR)

Downstream utility is reported in Table 12. The numbers are presented as diagnostic and not as a directional ranking between Iterations 2 and 3, because the §7 limitation that TSTR can reward mode collapse is non-trivially active in the present comparison: Iteration 2 generates 17,698 unique synthetic SKUs against Iteration 3's 105,732, and the smaller-vocabulary generator produces an easier next-item prediction task by virtue of its narrower output space.

Table 12. Preliminary downstream utility (single seed). HR@10 and NDCG@10 higher is better. The OOV rate differs across iterations because the shared-vocabulary filter (Section 7.4) intersects with the synthetic SKU set produced by each generator.

	Iter 2			Iter 3		
Condition	HR@10	NDCG@10	OOV	HR@10	NDCG@10	OOV
TSTR-T	0.412	0.379	0.725	0.073	0.062	0.547
TSTR-M	0.009	0.007	0.725	0.009	0.007	0.547
TRTR	0.370	0.327	0.725	0.214	0.186	0.547

Two readings of the table.

The transformer dominates Markov on TSTR by a large margin in both iterations. TSTR-T HR@10 is 0.412 vs. 0.009 for Iteration 2 and 0.073 vs. 0.009 for Iteration 3, ratios of 46× and 8× respectively. The directional pre-registered claim (transformer TSTR > Markov TSTR) holds in both iterations.

The cross-iteration TSTR ranking inverts the bias-group ranking Iteration 2, the mode-collapsed iteration scores TSTR-T HR@10 of 0.412, above its own TRTR baseline of 0.370. Iteration 3, the over-diffuse iteration with near-zero ΔGini, scores 0.073, well below its TRTR baseline of 0.214. The fact that Iteration 2 exceeds its TRTR reference is itself a diagnostic signal that a generator should not be more useful for training a recommender than the real data it imitates. The most parsimonious explanation is that Iteration 2's synthetic data presents a substantially easier classification task than the real data does, by virtue of its restricted output vocabulary, and the recommender exploits this to reach an artificially high HR@10. Iteration 3 does the opposite: by emitting a broader and flatter SKU distribution, it produces a harder task than the real data, and the recommender underperforms TRTR accordingly.

This does not imply that Iteration 2 is the better generator, but that TSTR-T as a single scalar is insufficient to rank generators that differ in effective output vocabulary. The final §8 will pair TSTR with a coverage-controlled variant so that vocabulary effects can be separated from genuine downstream-utility differences.

Table 13. Status of the §6 pre-registered pass conditions on the preliminary single-seed data.

Condition	Status
Iter 2: action/temporal not regressed	*Met*
Iter 2: coverage and ΔGini up	*Met* (vs. qualitative Iter 1)
Iter 3: action/temporal not regressed	*Met* (improved)
Iter 3: bias group ≥ Iter 2	*Mixed* (Gini ↓, popJSD ↑)
Iter 3: TSTR > Iter 2	*Not met* (vocabulary-confounded; see §8.3)
Cross-arch: transformer > Markov	*Met on session-shape and TSTR*

8.4 Pre-registered pass conditions

9 Discussion

9.1 Interpretation of Findings

To be written against final results. The preliminary picture is a coverage-vs-utility trade-off between Iterations 2 and 3, complicated by vocabulary-scope confounds in TSTR. The final §9.1 will tie this to each research sub-question:

- SQ1 (schema/interface contract): whether the schema held up under generation.
- SQ2 (architectural benefits over Markov): whether the structural priors produced the predicted benefits.
- SQ3 (comparison portfolio): whether the single-Markov baseline was sufficient.
- SQ4 (falsifiability and reproducibility): whether the pre-registered pass conditions actually functioned as a falsification mechanism.

9.2 Validity

Internal validity. The internal-validity threat in the present results is the single-seed protocol: with $n = 1$ no claim can be made about whether the observed differences between iterations and against the Markov baseline reflect architectural effects or seed-to-seed sampling variation. The five-seed protocol of Section 7.1 addresses this and will be run for the final results. Until then, all numerical comparisons in Chapter. 8 should be taken as directional rather than confirmed. A secondary threat is the cross-iteration evaluation-context drift documented in the Chapter. 8 status note: Iteration 1 was evaluated under an earlier preprocessing configuration and is being retrained under the current one, so its inclusion in the final comparison takes at least 1 more day from writing. Two specific anomalies surfaced in the preliminary numbers are flagged: the KS(δ_t) value of 0.251 shared to four decimal places across both transformer iterations and the Markov baseline, which is most plausibly an artefact of the temporal-head discretization grid, and Iteration 2's TSTR-T HR@10 (0.412) exceeding its own TRTR baseline (0.370) on a vocabulary-restricted evaluation, which is consistent with the Chapter. 7 mode-collapse-rewards-TSTR limitation but warrants a explicit confirmation under the final protocol.

External validity. The Synerise dataset originates from a single retail e-commerce platform, and the schema, hard constraints, and validity rules reported here represent retail conventions of the platform's tracking implementation. The Chapter. 5.3 limitations section already scoped this. The preliminary results do not extend this scope. Particularly, the finding that approximately one-third of real training purchases lack prior same-SKU cart events (Section 5, Figure 8) is a property of this dataset's interaction logging, and not a domain-agnostic claim about shopping behavior. The corresponding soft treatment of the exposure constraint should be re-derived rather than transferred when applying this methodology to different domains. Generalization to other session-based interaction settings such as news, media, music

streaming, and social feeds would require re-examination of the valid-transition matrix and the exposure-prerequisite distribution from domain-appropriate training data.

Construct validity. The construct-validity argument of Section 7.5 held that each fidelity metric was chosen to catch a separable failure mode, so that a generator could pass one and fail another. The preliminary results provide evidence that this separation exists, namely Iteration 3 passes ΔGini (0.014) while failing popularity JSD (0.487), and Iteration 2 passes neither but does so in a different direction (concentrated rather than diffuse). A single aggregate distance would have collapsed these into one number and hidden the failure-mode distinction that motivated Iterations 2 and 3 in the first place. The construct-validity argument holds for the bias group. The most consequential construct-validity question raised by the results is for downstream utility, namely that TSTR-T as a single scalar is insufficient to rank generators that differ in effective output vocabulary, because vocabulary scope confounds the comparison. This was anticipated as a Chapter. 7 limitation but appears empirically active in the preliminary data, and a coverage-controlled TSTR variant, evaluating each generator's TSTR on a common, externally-fixed vocabulary rather than each generator's own synthetic vocabulary,is identified as a methodological extension the final results should include.

Ecological validity. The ecological-validity scoping of Section 7.5 is unchanged by the preliminary results. Offline evaluation supports claims about static log fidelity, hard-constraint validity rates, and training-set utility for a downstream recommender. It does not support claims about generator performance under live deployment with real feedback dynamics, exploration-exploitation interactions, or user adaptation. No live A/B testing is performed, and none of the results in Chapter. 8 should be read as predictive of deployment behavior. The question of whether a generator that produces high-fidelity offline sessions also produces useful synthetic traffic for live system testing remains open and is identified as future work.

9.3 Reliability

n=1 Results cannot support reliability claims, the final report will consist of five-seed variance and address this.

9.4 Limitations

The following limitations should temper the interpretation of the results. Data-side limitations are discussed in Section 5.3; this subsection covers limitations of the model and evaluation.

- **Comparison scope.** The portfolio is restricted to a single Markov baseline. Stronger baselines (GRU/LSTM-based or GAN-based generators) are identified as future work.
- **Single dataset.** Results are reported on Synerise only; cross-dataset generalization is not tested.
- **Offline-only evaluation.** As discussed under ecological validity, no live deployment or A/B testing is performed.
- **Training cost.** Transformer training cost limits the number of ablations and iterations that could be run within the project timeline.
- **Throughput.** Autoregressive generation is sequential at inference time. If the generator cannot keep pace with the arrival simulator at full scale, the pipeline may stall; batched generation is identified as a fallback.
- **Teacher-forced training.** The model is trained on ground-truth prefixes but generates from its own prior outputs at inference time. This exposure-bias gap can produce drift over long rollouts. Standard mitigations such as scheduled sampling[1] or sequence-level training were not pursued due to time constraints.

- **Bias inheritance.** The generator inherits the popularity skew and temporal selection effects of the training data. Bias metrics are reported, but reporting does not eliminate the bias.
- **Seasonal non-stationarity.** E-commerce traffic and item popularity vary sharply across the year (Black Friday, Christmas, summer vacations). The test split sits inside one such regime (early December) by construction (Section 5), so held-out performance conflates generation quality with seasonal extrapolation from the June–November training window.
- **TSTR can reward mode collapse.** TSTR scores depend on the synthetic training set being a representative learning signal. A generator that collapses onto a small head of popular SKUs produces an artificially easy next-item prediction task, on which the downstream recommender can score highly despite the upstream failure. This is the principal reason the bias group (item coverage, popularity JSD, ΔGini) is reported alongside TSTR rather than absorbed into a single utility number.
- **Shared-vocabulary TSTR comparability.** Seasonal item drift means different generators may sample different SKU subsets, so vocabularies diverge across runs. To keep TSTR-T, TSTR-M, and TRTR comparable, a shared vocabulary filter is applied across all three. The trade-off is that absolute TSTR scores are computed on a reduced vocabulary and are not directly comparable to TSTR numbers reported elsewhere in the literature; only the within-thesis comparison is intended to be read as a like-for-like test.

9.5 Ethical Reflection

10 Conclusion

10.1 Summary of Contributions

10.2 Implications

10.3 Recommendations and Future Work

Acknowledgments

References

[1] Bengio, S., Vinyals, O., Jaitly, N., and Shazeer, N. Scheduled sampling for sequence prediction with recurrent neural networks, 2015.

[2] Bobadilla, J., Gutiérrez, A., Yera, R., and Martínez, L. Creating synthetic datasets for collaborative filtering recommender systems using generative adversarial networks. *Knowledge-Based Systems* (2023).

[3] Bountouridis, D., Harambam, J., Makhortykh, M., Marrero, M., Tintarev, N., and Hauff, C. Siren: A simulation framework for understanding the effects of recommender systems in online news environments. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAT*)* (2019).

[4] Brown, S. The c4 model for software architecture. https://static.simonbrown.je/pth2024-c4.pdf, 2024. Presentation slides, accessed 2026-05-11.

[5] Chaney, A. J. B. Recommendation system simulations: A discussion of two key challenges. *arXiv preprint arXiv:2109.02475* (2021).

[6] Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., and Bharath, A. A. Generative adversarial networks: An overview. *IEEE Signal Processing Magazine 35*, 1 (2018), 53–65.

[7] Cui, Y., Jia, M., Lin, T.-Y., Song, Y., and Belongie, S. Class-balanced loss based on effective number of samples, 2019.

[8] Ekstrand, M. D., Chaney, A., Castells, P., Burke, R., Rohde, D., and Slokom, M. Simurec: Workshop on synthetic data and simulation methods for recommender systems research. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)* (2021), pp. 803–805.

[9] Esteban, C., Hyland, S. L., and Rätsch, G. Real-valued (medical) time series generation with recurrent conditional gans, 2017.

[10] European Parliament and Council of the European Union. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data (general data protection regulation). *Official Journal of the European Union L 119* (2016), 1–88.

[11] Figueira, A., and Vaz, B. Survey on synthetic data generation, evaluation methods and gans. *Mathematics 10*, 15 (2022), 2733.

[12] Gu, J., Bradbury, J., Xiong, C., Li, V. O. K., and Socher, R. Non-autoregressive neural machine translation, 2018.

[13] Halfaker, A., Keyes, O., Kluver, D., Thebault-Spieker, J., Nguyen, T., Shores, K., Uduwage, A., and Warncke-Wang, M. User session identification based on strong regularities in inter-activity time, 2019.

[14] Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
[15] Hurley, N. P., and Rickard, S. T. Comparing measures of sparsity, 2009.
[16] Jakomin, M., Pevec, D., Kononenko, I., and Bosnić, Z. Generating interdependent data streams for recommender systems. *Expert Systems with Applications 112* (2018), 473–484.
[17] Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax, 2017.
[18] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128. INRIA Research Report inria-00514462v2.
[19] Jordon, J., Szpruch, L., Houssiau, F., Bottarelli, M., Cherubin, G., Maple, C., Cohen, S. D., and Weller, A. Synthetic data: What, why and how? *arXiv preprint arXiv:2205.03257* (2022).
[20] Jr., F. J. M. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association 46*, 253 (1951), 68–78.
[21] Kang, W. C., and McAuley, J. Self-attentive sequential recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)* (2018).
[22] Lin, J. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory 37*, 1 (1991), 145–151.
[23] Lu, Y., Shen, A., Wang, H., Wang, X., van Rechem, C., and Fu, W. Machine learning for synthetic data generation: A review. *arXiv preprint* (2024).
[24] Lucherini, E., Sun, M., Winecoff, M., and Narayanan, A. T-recs: A simulation tool to study the societal impact of recommender systems. *arXiv preprint arXiv:2107.08959* (2021).
[25] McInerney, J., Brost, B., Chandar, P., Mehrotra, R., and Carterette, B. Accordion: A trainable simulator for long-term interactive systems. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys)* (2021).
[26] Meng, Z., McCreadie, R., Macdonald, C., and Ounis, I. Exploring data splitting strategies for the evaluation of recommendation models, 2020.
[27] Menon, A. K., Jayasumana, S., Rawat, A. S., Jain, H., Veit, A., and Kumar, S. Long-tail learning via logit adjustment. *arXiv preprint arXiv:2007.07314* (2021).
[28] Petrov, A. V., and Macdonald, C. Generative sequential recommendation with gptrec. *arXiv preprint arXiv:2306.11114* (2023).
[29] Rodriguez-Hernandez, M. C., Ilarri, S., Trillo-Lado, R., and Hermoso, R. Datagencars: A generator of synthetic data for the evaluation of context-aware recommendation systems. *Pervasive and Mobile Computing 38* (2017), 516–541.
[30] Stavinova, E., Grigorievskiy, A., Volodkevich, A., Chunaev, P., Bochenina, K., and Bugaychenko, D. Synthetic data-based simulators for recommender systems: A survey. *arXiv preprint arXiv:2206.11338* (2022).
[31] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, H. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)* (2019).
[32] Synerise. Synerise recsys challenge 2025. https://github.com/Synerise/recsys2025, 2025. GitHub repository, accessed 2026-05-11.
[33] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need, 2023.
[34] Williams, R. J., and Zipser, D. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation 1*, 2 (1989), 270–280.
[35] Zhao, X., Xia, L., Zou, L., Liu, H., Yin, D., and Tang, J. Usersim: User simulation via supervised generative adversarial network. In *Proceedings of The Web Conference (WWW)* (2021).

A Model Architecture Diagrams

This appendix collects the architecture diagrams that are referenced from Sections 6.2 and 6.3. Figure 12 gives the component-level view of the generation runtime that hosts all three iterations. The remaining figures show the item-prediction branch for each iteration; all three share the same transformer backbone, action head, temporal head, and cross-session memory and differ only in how the next SKU is predicted from the shared hidden state.

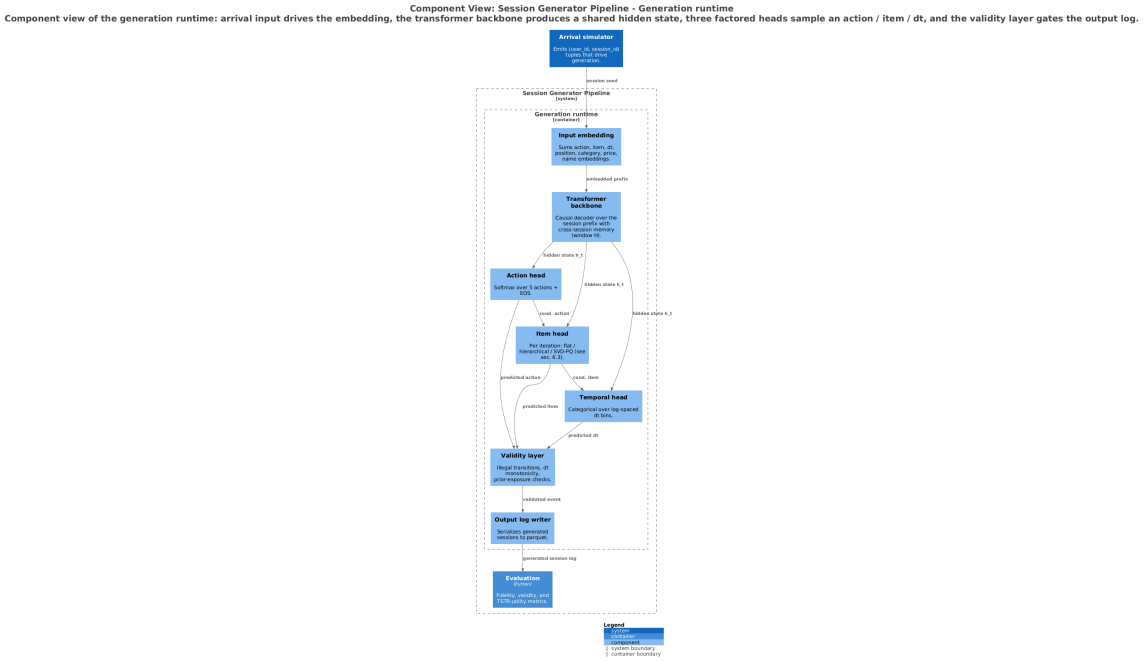

Fig. 12. Component view of the generation runtime (C4 component diagram). The arrival simulator drives the input embedding; the transformer backbone produces a shared hidden state; three factored heads sample an action, an item, and an inter-event time; the validity layer gates the output log. Per-iteration item-head variants are described in Section 6.3; the validity layer is specified in Section 6.4.

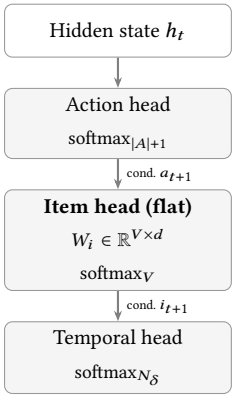

Fig. 13. Iteration 1. A single V-way softmax predicts the SKU directly from h_t, conditioned only on the sampled action.

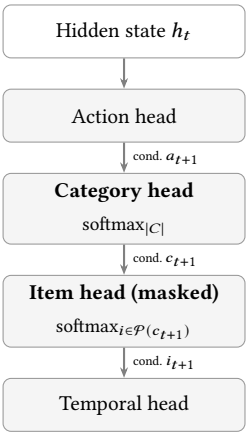

Fig. 14. Iteration 2. A category head intervenes between h_t and the item softmax; the item softmax is masked to the SKU pool of the sampled category.

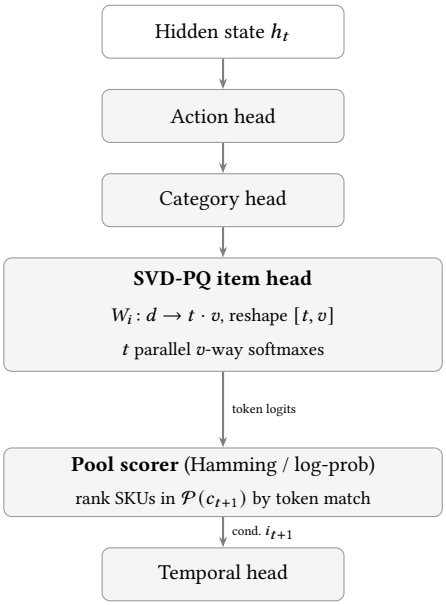

Fig. 15. Iteration 3. The item head emits t parallel v-way token softmaxes; an offline SVD-PQ codebook maps each SKU to a token tuple, and an in-pool scorer maps the token logits back to a single SKU at inference time.

B Additional Experimental Details

The fidelity metric suite referenced from Section 7.2 is reproduced in full in Table 14.

[TODO – full hyperparameter table, training curves, hardware utilization notes Take graphs provided by WandB.]

Table 14. Fidelity metric suite. Distributional metrics compare generated sessions to the training distribution; behavioral-rate metrics compare session-level event composition; bias metrics are informational and diagnose item-side collapse against a size-matched real subsample. Lower is better for all metrics except where indicated. All metrics are implemented in `evaluation/fidelity.py`.

Group	Metric	Range	Description
Distributional	JSD(action distribution)	$[0, 1]$	Jensen–Shannon divergence between real and generated action-type frequencies (5 event types). Marginal-level behavioral mix [22].
	KS(session length)	$[0, 1]$	Two-sample Kolmogorov–Smirnov statistic on session length distributions. Engagement-shape match.
	L1(action bigram matrix)	$[0, 2]$	L1 distance between normalized real and generated action bigram transition matrices. Sequential structure of actions, not just marginals.
	KS(temporal δ_t)	$[0, 1]$	Two-sample KS on inter-event delta-time distributions in seconds. Pacing realism.
	Sample diversity (Jaccard ratio)	ratio, 1.0 ideal	Mean pairwise Jaccard distance over item sets in synthetic sessions, normalized by the same statistic on real sessions. Reported as a ratio so the sparse-catalog Jaccard floor cancels; < 1 means synthetic sessions are too repetitive across users, > 1 means too diffuse.
Behavioral rate	Δ conversion rate	$[0, 1]$	Absolute difference between the fraction of synthetic and real sessions containing at least one `product_buy`. Funnel endpoint match.
	Δ cart abandonment rate	$[0, 1]$	Absolute difference between the fraction of sessions containing `add_to_cart` but no `product_buy`. Mid-funnel drop-off.
Bias (informational)	Item coverage	ratio, 1.0 ideal	Unique synthetic SKUs divided by unique SKUs in the size-matched real subsample. Diagnoses head-only generation.
	Popularity JSD	$[0, 1]$	JSD between synthetic and matched-real per-SKU popularity distributions over the union of SKU keys (no top-K truncation).
	Δ Gini	$[0, 1]$	Absolute difference between the Gini coefficients of the synthetic and matched-real SKU frequency distributions. Concentration parity.

C Additional Results

[TODO – extended fidelity tables, full transition matrices, per-seed results, additional qualitative examples.]

D Demonstrator Pipeline Reference

[TODO – brief reference for the configuration-driven demonstrator script: inputs, outputs, supported generator types, sample run on a held-out subset of the Synerise dataset.]